Lecture 3

Stream ciphers in practice

3.1 LFSRs

Linear-feedback shift registers(LFSRs) are often used as a building block for constructing a PRG, as they are efficient to implement in hardware and generate outputs that have good statistical properties. An LFSR consists of n memory cells $(s_0, \ldots, s_{n-1})$ each containing 1 bit of data, the bank of all n cells being the register, and a feedback function defined by a set of n boolean feedback coefficients $c_0, \ldots, c_{n-1}$. The feedback function XORs together a subset of the memory cells based on the feedback coefficients. The LFSR starts with an initial state at time $t = 0$, when t is incremented then the value in $s_0^{(t)}$ is outputted. As t increments, all the values in the memory cells are moved down one and $s_{n-1}^{(t+1)}$ is generated using the feedback function. More formally the state at time $t+1$ is defined from time t as:

$$s_i^{(t+1)} \leftarrow s_{i+1}^{(t)}, \qquad i \in [0, \ldots, n-2]$$
$$s_{n-1}^{(t+1)} \leftarrow \bigoplus_{j=0}^{n-1} c_j s_j^{(t)} \ .$$

As the feedback function only generates the new value for s_{n-1}, the first n output bits of the LFSR will be exactly the input.

Example Consider the degree-4 LFSR defined by the coefficients $c_0 = 1, c_1 = 1, c_2 = 0, c_3 = 0$ - i.e as the time increments the value for $s_3^{(t+1)}$ will be $s_0^{(t)} \oplus s_1^{(t)}$. Let the initial state be $(s_3, s_2, s_1, s_0) = (0, 1, 1, 0)$, then the states for each time step will be:

$$\begin{aligned}
t = 0; &\quad s = (0, 1, 1, 0) \\
t = 1; &\quad s = (1, 0, 1, 1) \\
t = 2; &\quad s = (0, 1, 0, 1) \\
t = 3; &\quad s = (1, 0, 1, 0) \\
t = 4; &\quad s = (1, 1, 0, 1) \\
t = 5; &\quad s = (1, 1, 1, 0) \\
t = 6; &\quad s = (1, 1, 1, 1) \\
t = 7; &\quad s = (0, 1, 1, 1) \\
t = 8; &\quad s = (0, 0, 1, 1)
\end{aligned}$$

How long can the output be? A degree-n LFSR has 2^n possible states - corresponding to the possible values of the bits in the register. If the state of the LFSR is ever the all 0 vector, then it will remain in that state forever. If it never hits the all 0 vector, then it will always cycle periodically - if a state repeats then the sequence will repeat, as there is only a finite number of states it must

eventually repeat a state. An LFSR is said to be of maximum length if it cycles through all $2^n - 1$ nonzero states before repeating, this is dependent entirely on the feedback coefficients. It is possible to choose coefficients such that this happens - specifically such that the polynomial defined by the coefficients is primitive - understanding and proving this is beyond the scope of these lectures.

Building a steam cipher Consider the PRG G defined as follows: let L be some predetermined maximum length degree-n LFSR, G takes as input some n bit key k (note this can never be the all 0 key), this is then the initial state of L, G runs L for as many timesteps as need and outputs the 'output' bit from each timestep. Unfortunately this isn't quite secure.

Firstly, lets assume that the coefficients are known (Kerchkhoff's principle) - then the first n bits of the output will reveal the initial state (i.e. the key). If we set G to ignore the first n bits of the output of L, still any n consecutive bits will allow an attacker to reproduce the remaining bits of the output.

Even if extend the key to be $2n$ bits, where the first n bits represent the coefficients, and the remaining n bits form the initial sate, it is possible to extract the coefficients from the first $2n$ bits of the output. The first n output bits $(y_0, \ldots, y_{n-1})$ are the initial state, and then the second n output bits $(y_n, \ldots, y_{2n-1})$ can be used to generate n linear equations with n unknowns (the coefficients) as follows

$$\begin{aligned} y_n &= c_{n-1}y_{n-1} \oplus \ldots \oplus c_0 y_0 \\ &\vdots \\ y_{2n-1} &= c_{n-1}y_{2n-2} \oplus \ldots \oplus c_0 y_{n-1} \ . \end{aligned}$$

If L is a maximum length LFSR then these are linearly independent, and thus can be solved efficiently using linear algebra.

3.1.1 Nonlinearity

In order to address the issue with LFSRs, we need to introduce some element of non-linearity.

Nonlinear feedback function The first option is to replace the linear feedback function with some nonlinear function g. Where the output of g is 0 (or 1) with probability roughly 1/2 over the inputs - this is then known as a FSR. The stream cipher Trivium uses this method.

Nonlinear output Rather than the output of the LFSR being the rightmost bit, the output could be some nonlinear function of the entire register. This method is used by the stream cipher 'snow' that encrypts 3GPP phone traffic.

Nonlinear combiner An alternative is to run several LFSRs in parallel and combine them in a nonlinear fashion. The LFSRs do not need to have the same degree.

Warning! The nonlinear function needs to always be balanced. Consider the nonlinear combiner $a \wedge b \wedge c$, where a, b, c are the outputs of three LFSRs, then this will output 0 approximately 7/8 of the time.

3.2 Cryptosystems

3.2.1 Trivium

Trivium uses three nonlinear FSRs denoted A (which has degree 93), B (which has degree 84) and C (which has degree 111). The output of each FSR is the XOR of the right-most bit and on additional bit from within the register, with the output of Trivium being all three of these outputs XORed together.

The new leftmost bot of each FSR is computed as a function of one bit from the same FSR and a subset of the bits of another FSR - making the feeback function nonlinear. Trivium takes as input an 80-bit key and and 80-bit initialisation vector (IV), the key is loaded into the 80 leftmost bits of A, and the 80 bit IV is loaded into the 80 leftmost bits of B. All remaining bits are set to 0, except for the three rightmost bits of C (which are set to 1). Rather than taking the output from the first tick, Trivium is run for $4 \cdot 288$ clock ticks and only from then is the output taken.

3.2.2 Salsa and ChaCha

We start by describing the underlying PRG. The PRG takes as input a 256-bit seed s and a 64-bit initalisation vector IV and has two main components a padding function `pad` and a permutation π. `pad` takes as input the seed, the IV and a 64-bit counter to form a 512-bit block. The permutation is based on three operations, addition (modulo 2^{32}, bitwise rotation and XOR. The PRG outputs bits in 512 bit blocks as needed, where the ith block r_i is computed as $h_i \leftarrow \texttt{pad}(s, i, IV)$ and $r_i \leftarrow \pi(h_i) \oplus h_i$.

3.3 Generating random bits in practice

We've seen how a PRG can stretch a random seed into a longer bit string that looks random. But this requires the underlying seed to be random. We will also often assume in cryptography that we have access to some source of randomness for generating keys or IVs. Random numbers are generated using a random number generator (RNG). A key difference between a PRG and a RNG is that entropy is continuously added to the internal state of the RNG. This source of entropy can come from a number of sources including keyboard events, mouse events or hardware interrupts.